实验心得体会

院系	计算机学院	班级	8
学号	24325288	实验名称	网络编程实验（实验1 UDP 通信程序设计）
学生	闫维岳		

本人承担的工作

我作为组长负责实验方案设计，并用 Python 实现了 UDP 客户端与服务器端程序（`udp_client.py`、`udp_server.py`）。具体包括：设计 JSON 报文格式（`type` / `seq` / `sent_ns`），客户端循环发送 100 个带序号的数据报，服务器端用集合统计丢失、重复、乱序与格式错误；同时编写了《Python 本机测试说明》，梳理本机回环与跨机（局域网）两种场景的测试流程。

遇到的困难及解决方法

第一个困难是 UDP 无连接、本身没有"发送结束"的语义，服务器端无法判断客户端何时发完。我采用自定义 `type="end"` 结束标记，并在收到首包后用 `settimeout(5)` 设置接收超时——超时未收到新包即结束本轮统计，这样即使结束标记丢失也不会让服务器永久等待。

第二个困难是服务器打印 `Listening` 后长时间没有输出，一度以为是程序卡死。后来明白这是 `recvfrom()` 在阻塞等待数据，属于正常现象，只需保持服务器运行、再启动客户端即可。

第三个困难是统计口径的确定：重复包与乱序包容易混淆。我最终约定：序号已出现在 `received` 集合中则计重复；新序号小于历史最大序号则计乱序；其余情况更新最大序号，并用集合差 `expected - received` 得到丢失序号。三个指标互不重叠，统计才清晰。

体会与总结

这次实验让我体会到 Python 的 `socket` 模块让 UDP 编程非常简洁：`sendto` / `recvfrom` 直接传 `(ip, port)` 元组，省去了 C++ 里繁琐的地址结构填充。更重要的是，我深刻理解了 UDP 的不可靠性——数据报可能丢失、乱序、重复，要让 UDP"可靠"，必须在应用层自己补上序号、超时、重传等机制。我在报文中预留的 `sent_ns` 时间戳，也为后续做延迟分析留了接口。此外，跨机测试时让服务端监听 `0.0.0.0` 并放行防火墙端口，让我对"绑定地址"的含义有了实际体会。